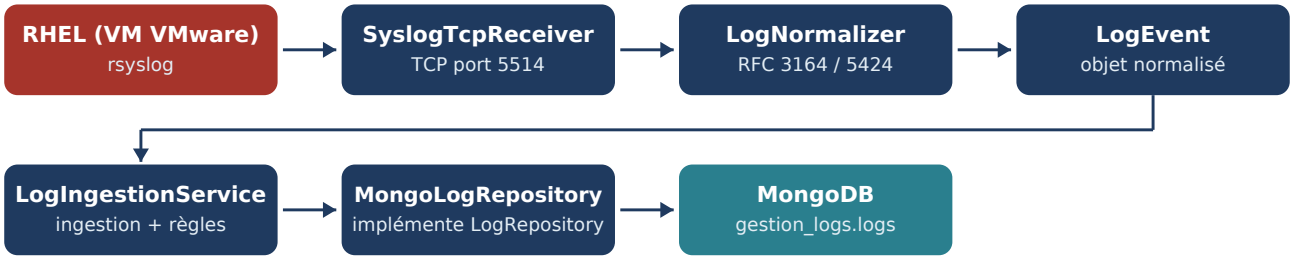
Guide d'installation et de mise en route

Du téléchargement sur GitHub à la chaîne RHEL → rsyslog → Java → MongoDB

Projet : Gestion intelligente des logs · 4ème année ingénierie informatique et réseaux
Public : membres de l'équipe qui installent le projet sur leur machine · Version 1.0 · 06/11/2026

1. Objectif et vue d'ensemble

À la fin de ce guide, votre machine reçoit les logs réels d'une machine **Red Hat Enterprise Linux (RHEL)**, les normalise en objets Java `LogEvent` et les enregistre dans **MongoDB**. Les étapes sont à suivre **dans l'ordre**. Chaque étape se termine par une vérification : ne passez pas à la suivante tant que la vérification n'est pas bonne.



Machine	Rôle	Ce qui y tourne
Windows (hôte)	Développement et réception	Eclipse, Java 25, Git, MongoDB, récepteur `SyslogTcpReceiver`
VM RHEL (VMware)	Source des logs	rsyslog configuré pour envoyer les logs vers Windows (TCP 5514)

Important
RHEL est la **seule** source de logs du projet. N'utilisez pas Ubuntu comme source. Chaque membre travaille avec sa propre machine Windows, sa propre VM RHEL et sa propre base MongoDB locale.

2. Prérequis

Composant	Version	Rôle	Téléchargement
JDK (Java)	**25** (ex. Temurin 25)	Compiler et exécuter	adoptium.net
Eclipse IDE for Java Developers	**2025-12 ou plus récent**	IDE + Maven intégré (M2E) + Git	eclipse.org/downloads
Git for Windows	récente	Récupérer le projet	git-scm.com
MongoDB Community Server	**9.0.4** (version du projet)	Base de données	mongodb.com/try/download/community
MongoDB Shell (mongosh)	récente	Vérifier les données	mongodb.com/try/download/shell
VMware Workstation + ISO RHEL	—	Machine source des logs	VMware / Red Hat Developer

Attention
Java 25 est intégré dans Eclipse à partir de la version **2025-12**. Avec la version 2025-09, il faut ajouter le patch « Java 25 Support for Eclipse 2025-09 » depuis le Marketplace. Prenez donc directement une version récente d'Eclipse.

À savoir
Maven n'a pas besoin d'être installé : on utilise celui d'Eclipse. Il est normal que la commande `mvn` ne soit pas reconnue dans CMD. Prévoyez aussi les droits administrateur sur Windows et, si possible, 16 Go de RAM (Windows + VM RHEL + MongoDB + Eclipse).

3. Installation de l'environnement (Windows)

Étape 1 : Java 25

- Installez le JDK 25 (installeur `.msi` Temurin ou Oracle JDK 25).
- Ouvrez un **nouveau** CMD et vérifiez :

```
java -version
javac -version
```

Résultat attendu
Les deux commandes affichent une version **25.x**.

- Dans Eclipse : **Window → Preferences → Java → Installed JREs → Add… → Standard VM**, choisissez le dossier du JDK 25, cochez-le comme JRE par défaut, puis **Apply and Close**.

Étape 2 : Git

- Installez Git for Windows (options par défaut, Git Credential Manager inclus), puis dans CMD :

```
git --version
git config --global user.name "Prénom Nom"
git config --global user.email "votre.email@exemple.com"
```

Résultat attendu
`git --version` affiche une version. Utilisez, pour `user.email`, l'adresse de votre compte GitHub.

Étape 3 : MongoDB et mongosh

- Installez **MongoDB Community Server 9.0.4** avec l'installeur `.msi` en mode **Complete**, en laissant coché « Install MongoDB as a Service » (service nommé `MongoDB`).
- Installez ensuite **mongosh** (installeur séparé).
- Vérifiez que le service tourne, puis testez la connexion :

```
sc query MongoDB
mongosh --eval "db.runCommand({ping:1})"
```

Résultat attendu
`STATE : 4 RUNNING` pour le service, et `{ ok: 1 }` pour le ping.

Attention
MongoDB doit rester accessible **uniquement en local** (127.0.0.1, configuration par défaut). N'ouvrez pas le port 27017 dans le pare-feu et ne l'exposez pas au réseau. Si le service est arrêté : `net start MongoDB` dans un CMD ouvert **en administrateur**.

4. Récupérer le projet depuis GitHub

- Vous devez avoir **accepté l'invitation** du dépôt envoyée par l'administrateur du groupe (dépôt privé).
- Choisissez un dossier de travail simple, par exemple `C:\Projets` (évitez les dossiers synchronisés par OneDrive).
- Clonez le dépôt, puis placez-vous sur la branche de travail :

```
cd C:\Projets
git clone <URL-DU-DEPOT>
cd <NOM-DU-DEPOT>
git checkout integration-logs
git pull
```

À savoir
Au premier `git clone`, **Git Credential Manager** ouvre une fenêtre pour vous connecter à GitHub. Si le mot de passe est refusé, utilisez un **Personal Access Token** (GitHub → Settings → Developer settings) à la place du mot de passe.

5. Importer et compiler dans Eclipse

- **File → Import… → Maven → Existing Maven Projects**, sélectionnez le dossier cloné, **Finish**. Attendez la fin du téléchargement des dépendances (barre de progression en bas à droite).
- Clic droit sur le projet → **Maven → Update Project…** (Alt+F5), cochez **Force Update of Snapshots/Releases**, **OK**.
- Clic droit sur le projet → **Run As → Maven build…**, dans **Goals** saisissez `clean package`, puis **Run**.

Résultat attendu
La console affiche **BUILD SUCCESS**.

Attention
Si vous obtenez `invalid target release: 25` : dans la fenêtre **Run Configurations** de ce Maven build, onglet **JRE**, choisissez le **JDK 25** (et non le JRE par défaut d'Eclipse).

Vérification des tests existants

Le projet contient des programmes de test à lancer avec **clic droit → Run As → Java Application** (il n'y a pas encore de tests JUnit) :

Classe	Résultat attendu
`LogNormalizerTest`	Normalisation RFC 3164 et RFC 5424 : tous les tests OK, `FIN DES TESTS`
`MongoConnectionTest`	`Connexion MongoDB OK` (MongoDB doit tourner)
`MongoLogRepositoryTest`	6 lignes `OK` puis `TOUS LES TESTS OK` (utilise la base séparée `gestion_logs_test`)

6. Réseau entre Windows et la VM RHEL

Vous avez besoin de **deux adresses IP**. Elles sont propres à votre machine et peuvent changer quand VMware ou la VM redémarre : revérifiez-les si les logs n'arrivent plus.

Machine	Commande	Quelle adresse retenir
Windows	`ipconfig`	IPv4 de la carte **VMware Network Adapter VMnet8** (réseau NAT), ex. `192.168.159.1`
RHEL	`ip -4 addr`	IPv4 de l'interface réseau (ex. `ens160`), ex. `192.168.159.128`

Dans la suite, `<IP-WINDOWS>` et `<IP-RHEL>` désignent ces deux adresses. Test de base depuis Windows :

```
ping <IP-RHEL>
```

À savoir
Les adresses `192.168.159.1` (Windows) et `192.168.159.128` (RHEL) sont celles de la configuration de référence du projet. Les vôtres seront probablement différentes.

7. Pare-feu Windows

Autorisez les connexions entrantes sur le port TCP **5514**, uniquement depuis votre VM. CMD ouvert **en administrateur** :

```
netsh advfirewall firewall add rule name="Gestion Logs Syslog 5514" dir=in action=allow ^
    protocol=TCP localport=5514 remoteip=<IP-RHEL>
```

À savoir
Au premier lancement du récepteur, Windows peut afficher une fenêtre d'autorisation pour Java : autorisez-la. Pour supprimer la règle plus tard : `netsh advfirewall firewall delete rule name="Gestion Logs Syslog 5514"`.

8. Configuration de RHEL (rsyslog)

Sur la VM RHEL, rsyslog est normalement déjà installé. Vérifiez-le, puis créez le fichier de redirection :

```
sudo systemctl status rsyslog
sudo vi /etc/rsyslog.d/90-gestionlogs.conf
```

Contenu du fichier (remplacez `<IP-WINDOWS>`) :

```
# Envoi de tous les logs vers le récepteur Java sur Windows
*.* action(type="omfwd"
           target="<IP-WINDOWS>"
           port="5514"
           protocol="tcp"
           action.resumeRetryCount="-1"
           queue.type="LinkedList"
           queue.size="10000")
```

Vérifiez la syntaxe, redémarrez rsyslog, puis donnez à la VM un nom de machine exploitable :

```
sudo rsyslogd -N1
sudo hostnamectl set-hostname rhel-votreprenom
sudo systemctl restart rsyslog
sudo systemctl status rsyslog
```

Résultat attendu
`rsyslogd -N1` ne signale aucune erreur de configuration, et le service est `active (running)`.

Pourquoi changer le hostname ?
Le nom de la machine apparaît dans le champ `hostname` des logs, qui servira aux filtres et à la détection d'anomalies. Dans nos premiers tests, il valait `192` : rsyslog ne garde que la partie avant le premier point du nom, ce qui s'est produit probablement parce que le nom de la VM commençait par une adresse IP. Choisissez un nom **sans point** et qui ne commence pas par un nombre.

9. Lancer la chaîne complète et la valider

9.1 Préparer Windows

```
sc query MongoDB
netstat -ano | findstr :5514
```

- `MongoDB` doit être `RUNNING`.
- La commande `netstat` ne doit **rien** afficher. Si une ligne apparaît, une ancienne instance du récepteur occupe déjà le port : remplacez `<PID>` par le **dernier nombre** de la ligne et exécutez `taskkill /PID <PID> /F`. Ne lancez jamais deux instances du récepteur.

9.2 Démarrer le récepteur

- Dans Eclipse : clic droit sur `SyslogTcpReceiver.java` → **Run As → Java Application**.

```
Demarrage du recepteur TCP sur le port 5514
En attente de messages Syslog...
```

> **Attention**
> Si la console affiche « MongoDB inaccessible... Arret du recepteur », le service MongoDB n'est pas démarré : voir étape 3.

9.3 Vérifier la joignabilité, puis envoyer un log depuis RHEL

```
timeout 3 bash -c 'cat < /dev/null > /dev/tcp/<IP-WINDOWS>/5514' && echo "port 5514 joignable"
logger -p user.err "TEST ERROR depuis RHEL"
```

Dans la console Eclipse, vous devez voir apparaître :

```
Connexion etablie depuis : /<IP-RHEL>
=== LOG NORMALISE ===
Hostname : rhel-votreprenom
Service : <votre utilisateur>
Severity : ERROR
Message : TEST ERROR depuis RHEL
...
[MONGODB] Log saved: TEST ERROR depuis RHEL
[ALERTE] Rule matched: Detect Error
```

9.4 Vérifier dans MongoDB

```
mongosh
use gestion_logs
db.logs.countDocuments()
db.logs.find().sort({receivedTimestamp:-1}).limit(3)
db.logs.find({severity:"ERROR"})
```

> **Résultat attendu**
> Le nombre de documents augmente. Chaque document contient `hostname`, `service`, `severity`, `message` et `sourceAddress` (l'IP de RHEL), et `receivedTimestamp` s'affiche en `ISODate(...)`.

9.5 Liste de contrôle finale

	Vérification
☐	`java -version` et `javac -version` affichent 25.x
☐	Le projet compile : `BUILD SUCCESS`
☐	`MongoConnectionTest` et `MongoLogRepositoryTest` : tous les tests OK
☐	Le récepteur démarre sans erreur et écoute sur le port 5514
☐	Un log envoyé avec `logger` depuis RHEL s'affiche normalisé dans la console
☐	Le même log est visible dans `gestion_logs.logs` avec `mongosh`

Paramètres de référence (valeurs par défaut du code)

Paramètre	Valeur	Où
Port TCP du récepteur	`5514`	constante `PORT` de `SyslogTcpReceiver`
Connexion MongoDB	`mongodb://localhost:27017`	`MongoConnection`
Base / collection	`gestion_logs` / `logs`	`MongoConnection`
Base des tests	`gestion_logs_test`	`MongoLogRepositoryTest`

10. Dépannage

Symptôme	Cause probable	Solution
`invalid target release: 25`	Maven utilise un JRE plus ancien	Run Configurations → onglet JRE → JDK 25
`com.mongodb` souligné en rouge	Dépendance non téléchargée	Maven → Update Project (Force Update) ; vérifier la connexion Internet
« MongoDB inaccessible… »	Service MongoDB arrêté	`net start MongoDB` (CMD administrateur)
`Address already in use` au démarrage	Une autre instance du récepteur tourne	`netstat -ano | findstr :5514` puis `taskkill /PID <PID> /F`
Aucun log reçu	IP Windows erronée ou changée, pare-feu, récepteur non lancé, rsyslog non redémarré	Refaire les étapes 6 à 8 ; relancer le test `/dev/tcp` de l'étape 9.3
`hostname = 192` dans les logs	Nom de la VM commençant par une IP	`hostnamectl set-hostname` (étape 8) puis redémarrer rsyslog
Logs en console mais rien dans MongoDB	Récepteur encore branché sur l'ancien stockage en mémoire	Vérifier que la console affiche `[MONGODB]` et non `[DATABASE]`
Test `/dev/tcp` impossible malgré pare-feu OK	SELinux bloque rsyslog	`sudo ausearch -m avc -ts recent` ; si refus : `sudo semanage port -a -t syslogd_port_t -p tcp 5514`
Les logs s'arrêtent après un redémarrage	L'IP de la VM ou de Windows a changé	Reprendre l'étape 6 et corriger `target=` dans le fichier rsyslog
`Authentication failed` avec Git	Invitation non acceptée ou mot de passe refusé	Accepter l'invitation ; utiliser un Personal Access Token

11. Annexe : travailler en équipe avec Git

Branche	Usage
`main`	Version stable, relue par l'administrateur (pas de commit direct)
`integration-logs`	Branche d'intégration actuelle du pipeline logs → MongoDB
`feature/<sujet>`	Une branche par fonctionnalité, ex. `feature/anomaly-detection`

```
git checkout integration-logs
git pull
git checkout -b feature/anomaly-detection
# ... travail et tests ...
git status
git add chemin/du/fichier.java
git commit -m "feat: ajout du calcul du z-score"
git push -u origin feature/anomaly-detection
```

- Ouvrez ensuite une **Pull Request** sur GitHub vers `integration-logs` ; l'administrateur la relit et la fusionne.
- Ne versionnez jamais : le dossier `target/`, les fichiers de sauvegarde `*.bak`, les mots de passe ou jetons.
- Avant de modifier une classe partagée (`LogRepository`, `SyslogTcpReceiver`, `pom.xml`), prévenez l'administrateur : une signature modifiée casse le travail des autres.